Pembahasan Singkat KSN 2021 Bidang Informatika

Daring, 12 November 2021

Soal KSN 2021 Bidang Informatika

	Judul Soal	Pembuat Soal	Disiapkan oleh
1A	Pertahanan Mandiri	Ashar Fuadi	Vio Albert Ferdinand
1B	Lautan Biner	Prabowo Djonatan	Prabowo Djonatan
1C	Mewarnai Bola	Abdul Malik Nurrokhman	Abdul Malik Nurrokhman
2A	Permutasi Mandiri	Steven Novaryo	Steven Novaryo
2B	Daratan Biner	Pikatan Arya Bramajati + Scientific Committee	Lie, Maximilianus Maria Kolbe
2C	Mengirim Bola	Abdul Malik Nurrokhman	Prabowo Djonatan

Statistik Soal

	Judul Soal	Pengumpulan	Rata-rata	Maks
1A	Pertahanan Mandiri	734	19.05	75
1B	Lautan Biner	711	23.66	56
1C	Mewarnai Bola	630	33.97	100
2A	Permutasi Mandiri	719	24.39	100
2B	Daratan Biner	780	36.76	100
2C	Mengirim Bola	786	27.71	73

Pertahanan Mandiri

- Diberikan sebuah string berukuran N yang terdiri atas karakter ‘A’, ‘B’, dan ‘?’
- Karakter ‘?’ dapat diubah menjadi karakter ‘A’ dan ‘B’
- Ada berapa banyak cara untuk mengubah semua karakter ‘?’ sehingga jumlah substring yang berukuran M adalah **tepat** K modulo 10^9+7.

Subsoal 2

- Kita coba semua string yang mungkin (bruteforce)
- Untuk setiap karakter ‘?’, akan ada dua kemungkinan karakter berbeda yang dapat dibuat
- Ada karena ada paling banyak N buah karakter ‘?’, maka ada paling banyak 2^N string berbeda yang dapat dibuat

- Total kompleksitas waktu: $O(2^N)$

Subsoal 3 (1/2)

- Definisikan dp(idx, sisa, c) sebagai banyaknya cara untuk mengubah karakter ‘?’ dari index ke-1 hingga idx sehingga banyaknya substring berukuran M adalah tepat sisa dan karakter pada index idx adalah c.
  - c bernilai 0 jika ‘A’ dan 1 jika ’B’
- Base case
  - Jika sisa < 0, maka terdapat 0 cara
  - Jika idx = 0 dan sisa = 0, maka terdapat 1 cara
  - Jika idx = 0 dan sisa > 0, maka terdapat 0 cara

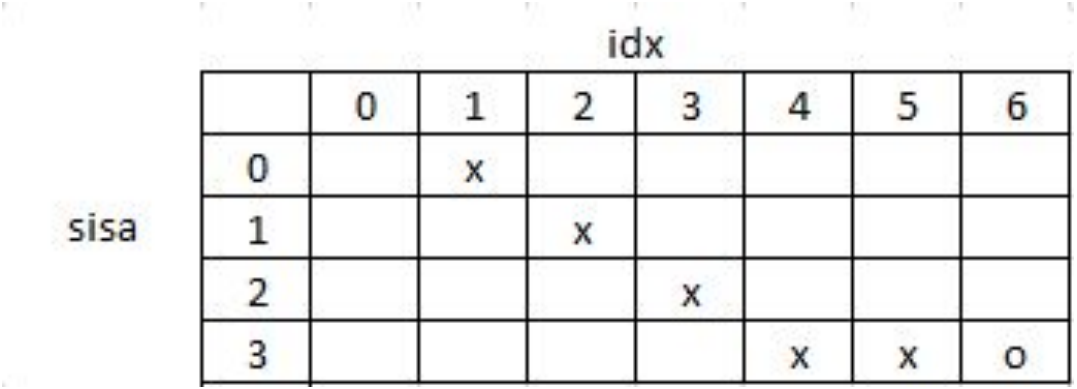
Subsoal 3 (2/2)

- Transisi
  - dp(idx, sisa, c) adalah penjumlahan dari
    - dp(idx-i, sisa, c xor 1) untuk setiap i < m dan semua karakter di antara idx-i dan i (inklusif) bukan (c xor 1).
    - dp(idx-i, sisa-(i-m), c xor 1) untuk setiap i >= m dan semua karakter di antara idx-i dan i (inklusif) bukan (c xor 1).
- Jawaban didapatkan dari dp(n, k, 0) + dp(n, k, 1)

- Jumlah state: O(NK), kompleksitas waktu transisi untuk tiap state: O(N)
- Total kompleksitas waktu: $O(N^2K)$

Solusi Penuh (1/2)

- Perhatikan bahwa untuk setiap transisi, kita dapat menyimpan sebuah prefix sum
  - Prefix sum horizontal: dp(idx-i, sisa, c^1) untuk setiap i < m dan semua karakter di antara idx-i dan i (inklusif) bukan (c^1)
  - Prefix sum diagonal: dp(idx-i, sisa-(i-m), c^1) untuk setiap i >= m dan semua karakter di antara idx-i dan i (inklusif) bukan (c^1)
- Berikut merupakan ilustrasi tabel dp dengan M = 2



Solusi Penuh (2/2)

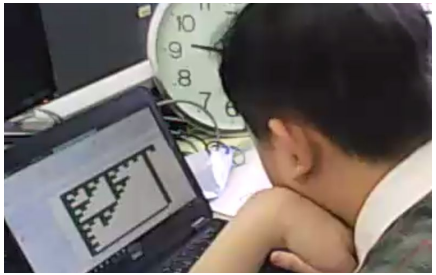
- Transisi dapat didapatkan dengan kompleksitas waktu O(1)
- Total kompleksitas waktu: O(NK)

Lautan Biner

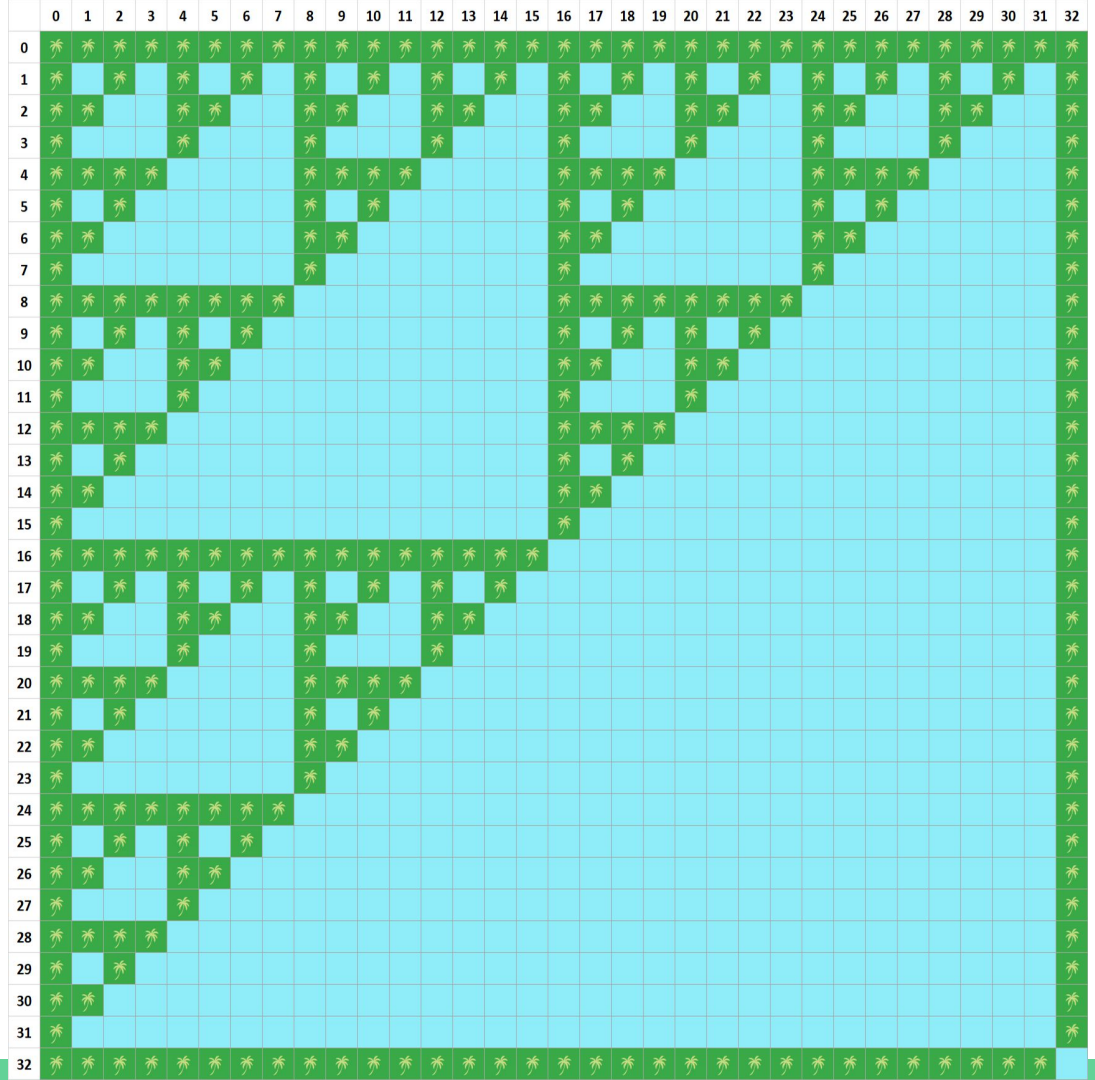
- Terdapat sebuah matriks N × M
- Baris i dan kolom j daratan ⇔ i & j = 0
- Ada Q pertanyaan:
  - Di submatriks (A, B) - (C, D) ada berapa banyak “pulau”
  - Pulau: kumpulan sel yang bisa dikunjungi tanpa melompat










0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32

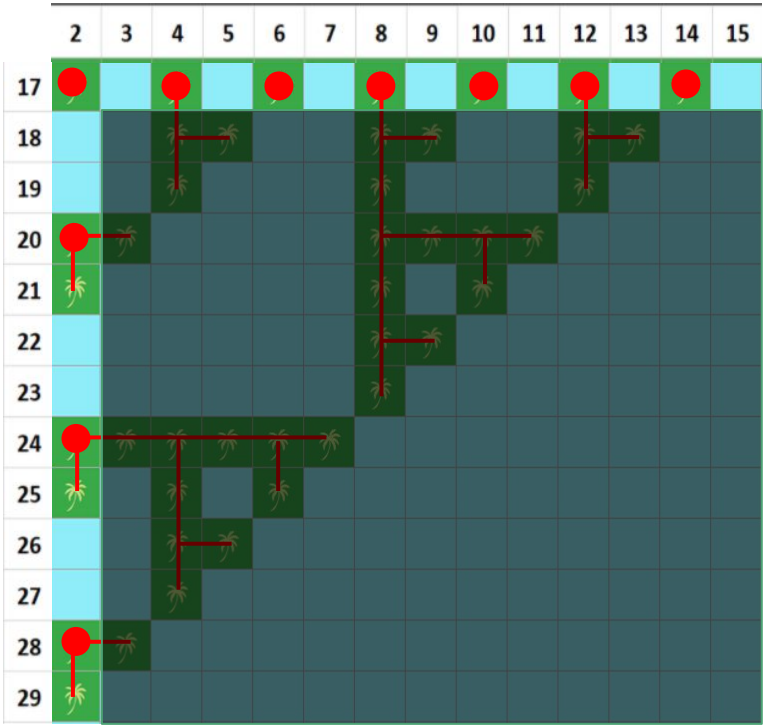
Terjemahkan ke Bentuk Graf

- Petak daratan adalah verteks
- Dua petak yang bersebelahan dibikin edge

Graf adalah Tree

- Dari petak daratan (r, c) (yang bukan (0, 0)) pasti bisa ke salah satu dari (r - 1, c) atau (r, c - 1) tetapi tidak keduanya
- Terdapat path unique dari (r, c) menuju (0, 0):
  - Misal LSB(x) adalah least significant bit dari x
  - LSB(r) ≠ LSB(c) karena jika sama maka AND tidak akan 0
  - WLOG LSB(r) < LSB(c)
    - (r - 1, c) pasti daratan, karena ubah r jadi r - 1 tidak akan merubah bit > LSB(r)
    - (r, c - 1) pasti bukan daratan, karena ubah c jadi c - 1 akan mengaktifkan semua bit < LSB(c)
- Karena semua petak daratan terhubung, dan hanya terdapat satu cara untuk berpergian antar petak daratan: graf adalah tree
- Subsoal 6 (A = B = 0) jawabannya adalah cetak 1 sebanyak Q kali

Cukup melihat tepi



Submatriks hanya ada satu baris

- Subsoal 7 (A = C)
- Kita coba hitung banyaknya daratan dari (A, 0) - (A, N - 1)
- Anggap semua N petak pada baris A adalah daratan
    - Kemudian iterasi dari bit A yang paling besar, dan buang petak yang diakibatkan bit tersebut

Contoh

A = 0101_2 N = 16



Bit 2:



Bit 0:



Pseudo code

```
count := N
for i largest to smallest:
  if i-th bit of A is on:
    count := count / 2^(i+1) + min(count % 2^(i+1), 2^i)
return count / 2^(lsb(A)) + 1
```

Solusi Penuh

- Jalankan subtask 7 sebanyak dua kali:
  - Untuk tepi atas dan tepi kiri
- Kompleksitas waktu akhir: O(Q(log N + log M))

Mewarnai Bola

- Soal interaktif.
- Ada N bola, diminta mewarnai bola sesuai dengan jenis bola.
- Boleh bertanya sebanyak Q kali, berapa banyak jenis berbeda dari [L, R].
- Apabila ada banyak cara pewarnaan, keluarkan pewarnaan mana pun.

Subsoal 1 dan 2

- Semua bahan yang sama memiliki nomor yang berurutan.
- A[i] = query(1, i).

Subsoal 3 dan 4

- Paling banyak 4 bahan berbeda.
- Misalkan kita sudah tahu semua warna dari [1, i] dan akan mencari A[i+1].
- Misalkan last(i, j) = nilai maksimum k <= i sehingga query(k, i) = j.
- Jika query(last(i, 2), i+1) = 2 maka:
  - Jika query(last(i, 1), i+1) = 1 maka A[i+1] = A[last(i, 1)].
  - Selain itu A[i+1] = A[last(i, 2)].
- Selain itu:
  - Jika query(last(i, 3), i+1) = 3 maka A[i+1] = A[last(i, 3)].
  - Selain itu A[i+1] = A[last(i, 4)].

Subsoal 5

- Ada N-1 atau N bahan berbeda.
- Cari R terkecil sehingga query(1, R) != R.
- Jika tidak ada maka semua bahan berbeda.
- Selain itu cari L terbesar sehingga query(L, N) = N-L.
- L dan R mempunyai bahan yang sama, sisanya berbeda semua.

Subsoal 6 dan 7

- Q = NlogN.
- Misalkan kita sudah tahu semua warna dari [1, i] dan akan mencari A[i+1].
- Cari x terkecil sehingga query(last(i, x), i+1) = x.
- Nilai x bisa dicari dengan binary search.
- A[i+1] = A[last(i, x)].

Permutasi Mandiri

- Diberikan sebuah permutasi A
- Dalam satu gerakan, bisa memilih 2 elemen bersebelahan dan menghapus elemen terbesar di antara keduanya.
- Cari banyak kemungkinan barisan berbeda yang dapat dibentuk.

Observasi

- Sebuah segmen [L, R] pada A dapat dihapus apabila $\min(A_{L-1}, A_{R+1}) < \min(A_L, A_{L+1}, \ldots A_R)$.
- Definisikan barisan B sebagai hasil dari rangkaian operasi yang dilakukan pada barisan A, dapat dilihat bahwa barisan B bisa didapatkan dengan melakukan penghapusan beberapa segmen valid pada barisan A.

A	9	8	1	4	3	2	7	5	6
B	9		1			2	7	5	

Subsoal 2

- Dynamic Programming menggunakan DP push bottom-up
- DP[i] adalah banyaknya penghapusan valid apabila elemen terakhir yang tidak dihapus adalah A_i.
- Transisi
  Untuk i, j, dan k dengan $i + 2 \leq j \leq k \leq N$ dan $\min(A_i, A_j) < \min(A_{i+1}, A_{i+2}, \ldots A_j)$, lakukan dp[k] += dp[i].
  Dengan kata lain kita ingin menghapus semua elemen di antara indeks i dan j, serta tidak menghapus elemen dengan indeks dari j sampai k.
- Base case DP[i] = banyaknya $j \leq i$ sehingga $A_j \leq \min(A_1, A_2, \ldots A_j)$.
- Jawabannya adalah jumlah dari DP[i] untuk setiap i dengan $A_i \leq \min(A_i, A_{i+1}, \ldots A_N)$.

Kompleksitas O(N^3)

Subsoal 5

- Jika kita menetapkan i dan j, bisa dilihat bahwa kita melakukan operasi dp[k] += dp[i] untuk setiap $j \leq k \leq N$.
- Oleh karena itu, untuk optimasinya kita dapat menyimpan prefix sum dari dp[i].

Subsoal 5

- Sewaktu kita mengiterasi i
  - Lakukan dp[i] += dp[i-1].
    Mirip seperti prefix sum.
  - Kemudian untuk i, dan j dengan $i + 2 \leq j \leq N$ dan $\min(A_i, A_j) < \min(A_{i+1}, A_{i+2}, \ldots A_{j-1})$, lakukan dp[j] += dp[i].
    Karena operasi sebelumnya, kita seperti melakukan update dp[k] += dp[i] untuk $j \leq k \leq N$, untuk setiap j memenuhi.
- Base case dp[i] = apakah i memenuhi $A_i \leq \min(A_1, A_2, \ldots A_i)$.

Kompleksitas O(N^2)

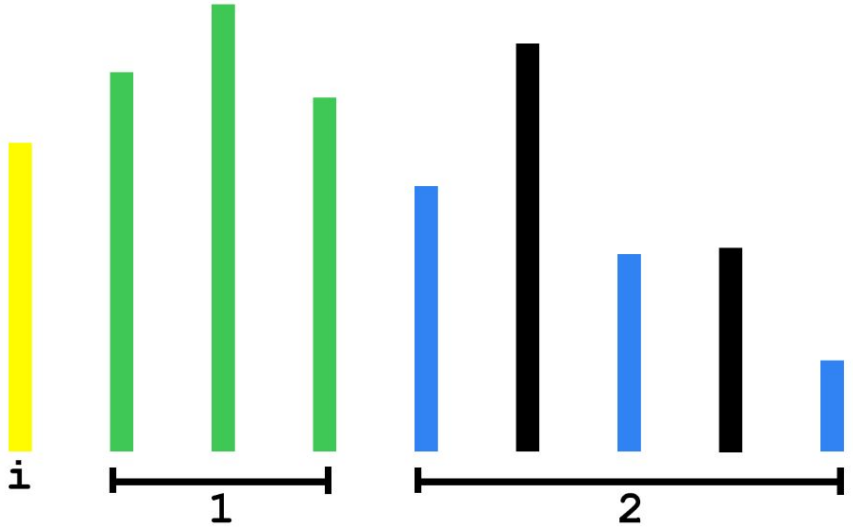
Subsoal 5 (Solusi Lainnya)

- DP(idx, erase) - banyak kemungkinan jika segmen yang dimulai pada indeks idx.
  - idx: posisi indeks yang diiterasi.
  - erase: apakah iterasi berikutnya akan menghapus segmen.
- Transisi:
  - DP(i, 0) = sum(DP(j, 1)) untuk semua j > i
  - DP(i, 1) = sum(DP(j, 0)) untuk semua j > i dan $\min(A_i, A_j) < \min(A_{i+1}, A_{i+2}, \ldots A_j)$.
    - Dalam artian kita menghapus semua elemen $(A_{i+1}, A_{i+2}, \ldots A_j)$.

Kompleksitas O(N^2)

Solusi Penuh

- Untuk suatu indeks i, kita dapat membagi indeks j yang memenuhi $\min(A_i, A_j) < \min(A_{i+1}, A_{i+2}, \ldots A_{j-1})$ menjadi 2 kasus, yaitu:
  1. $A_i < \min(A_{i+1}, A_{i+2}, \ldots A_j)$ atau
  2. $A_j < \min(A_i, A_{i+1} \ldots A_{j-1})$.



Solusi Penuh

- Misalkan nextSm(i) sebagai indeks k terkecil sehingga $A_k \leq \min(A_i, A_{i+1}, \ldots A_k)$. (dapat dicari menggunakan stack dalam O(N))
- Dapat dilihat bahwa j yang memenuhi kasus 1 berada pada range $i + 2 \leq j \leq$ nextSm(i).
- Sehingga kita dapat menggunakan range update atau line sweep.

Solusi Penuh

- Untuk kasus kedua perhatikan bahwa kumpulan indeks j yang memenuhi akan berbentuk **nextSm(i), nextSm(nextSm(i)), nextSm(nextSm(nextSm(i))), dst.** Dengan kata lain suatu indeks yang memenuhi merupakan nextSm dari indeks sebelumnya.
- Jika kita membentuk sebuah graf dengan indeks sebagai verteks dan nextSm(i) sebagai parent dari verteks i, dapat dilihat bahwa graf tersebut adalah forest.
- Dapat dilihat bahwa melakukan dp[j] += dp[i] pada kasus ini sama dengan melakukan update tersebut pada tree yang dibentuk.

Solusi Penuh

- Update path pada tree yang diperlukan umumnya dapat dilakukan dengan segment tree.
- Namun, karena di kasus ini semua children dari suatu verteks pada tree diakses duluan maka untuk setiap verteks u kita dapat menyimpan semua update dari anak verteks u dan u kemudian mengupdate parent-nya.

Pseudocode

```
for i in range(1, N):
        sweep_sum += prefix_sweep[i];
        dp[i] = dp[i-1] + tree_sum[i] + start_dp[i] + sweep_sum;
        // handle kasus (1)
        prefix_sweep[i+2] += dp[i];                              // untuk memulai suatu segment ditambahkan
        prefix_sweep[nextSmaller[i]] -= dp[i];                   // untuk mengakhiri suatu segment dikurangi
        // handle kasus (2)
        tree_sum[nextSmaller[i]] += tree_sum[i] + dp[i];         // menambahkan kontribusi dari verteks ini dan childnya ke parent

        if nextSmaller[i] > N:                                   // tidak terdapat indeks j dengan A[i] > A[j] dan i < j
                ans += dp[i];
```

Kompleksitas O(N)

Daratan Biner

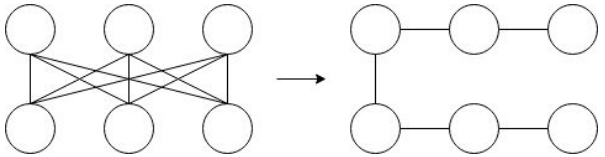
- Diberikan sebuah graf dengan N node
  - Node ke-i ada 2 bilangan, A[i] dan B[i]
- Ada edge dari node i ke node j jika A[i] xor A[j] > max(A[i], A[j])
- Untuk setiap i, cari sum(B[k]) untuk semua node k yang berada di satu *connected component* (CC) dengan i

Subsoal 2 & 3

- Construct graph dengan bruteforce
  - O(N^2)
- Lakukan BFS/DFS
  - Subsoal 2: BFS/DFS dari setiap node ➜ O(N^3)
  - Subsoal 3: BFS/DFS satu kali di masing-masing CC ➜ O(N^2)
- Untuk subsoal 3 juga dapat menggunakan disjoint set (UFDS/DSU)

Subsoal 6

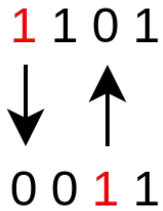
- Definisikan S[x] = { k | A[k] = x, 1 <= k <= N }
  - Awalnya, antar node dalam suatu himpunan tidak terhubung



- Iterasi semua pasangan nilai A ➜ O(Amax^2)
  - Misal pasangan sekarang adalah x dan y yang memenuhi x^y > max(x,y)
  - Hubungkan semua node di S[x] dengan S[y] ➜ tidak efisien, cukup butuh 1 saja yang terhubung
  - Hubungkan semua node dalam S[x], dalam S[y], dan 1 node dari S[x] ke S[y] ➜ sudah cukup
- Pastikan penghubungan semua node dalam suatu himpunan hanya terjadi maksimal 1 kali
- Kompleksitas: O(Amax^2 + N)

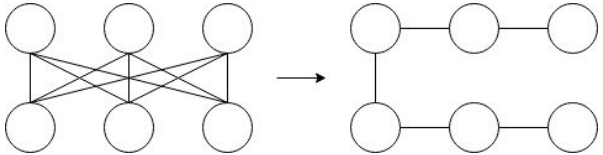
Subsoal 7

- Tidak bisa construct graph dengan bruteforce
- Observasi: x XOR y > max(x, y) jika (1<<MSB(x))&y=0 dan (1<<MSB(y))&x=0
  - MSB(x) = most significant bit, atau bit nyala paling kiri/paling besar dari x
  - & adalah bitwise and

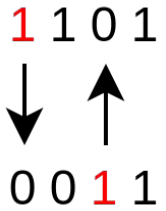


- WLOG x >= y. Syarat: x XOR y > x jika (1<<MSB(y))&x = 0
- Solusi: kelompokkan berdasarkan MSB, lalu jalankan solusi mirip subsoal 6

Subsoal 7



- Definisikan P[x] = { k | MSB(A[k]) = x, 1 <= k <= N }
  - Semua node k yang MSB dari A[k] adalah x
- Definisikan Q[x] = { k | (1<<y)&A[k] = 0, y < MSB(A[k]), 1 <= k <= N }
  - Semua node k yang bit ke-y nya mati dan bit tersebut bukan leading zero
- Untuk semua nilai x, hubungan P[x] ke Q[x] itu sama seperti hubungan antar S di Subsoal 6
- Kompleksitas waktu: O(N log Amax)



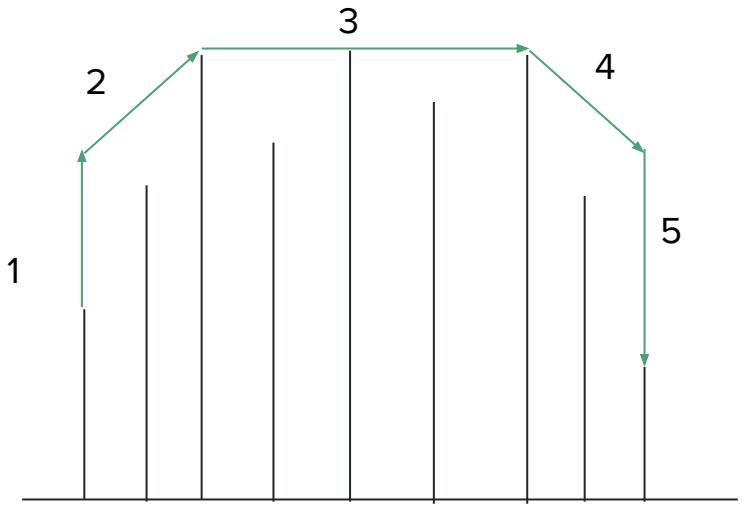
Mengirim Bola

- Ada bukit yang tingginya H_i dari kiri ke kanan
- Dalam satu gerakan:
  - Gerak kiri-kanan
  - Gerak atas-bawah
    - Ke atas cost-nya 4
    - Tetap cost-nya 2
    - Ke bawah cost-nya 1
- Ada Q pertanyaan
  - cost minimum dari puncak S ke puncak T

Observasi Greedy

- Semua gerakan ke atas bisa dilakukan di awal
- Semua gerakan ke bawah bisa dilakukan di akhir
- Proof pakai swapping arguments: Misal ada solusi yang tidak demikian
  - Turun-tetap ⇒ tetap-turun: tidak ada ruginya
  - Tetap-naik ⇒ naik-tetap: tidak ada ruginya
  - Naik-turun / turun-naik ⇒ tetap-tetap: lebih untung

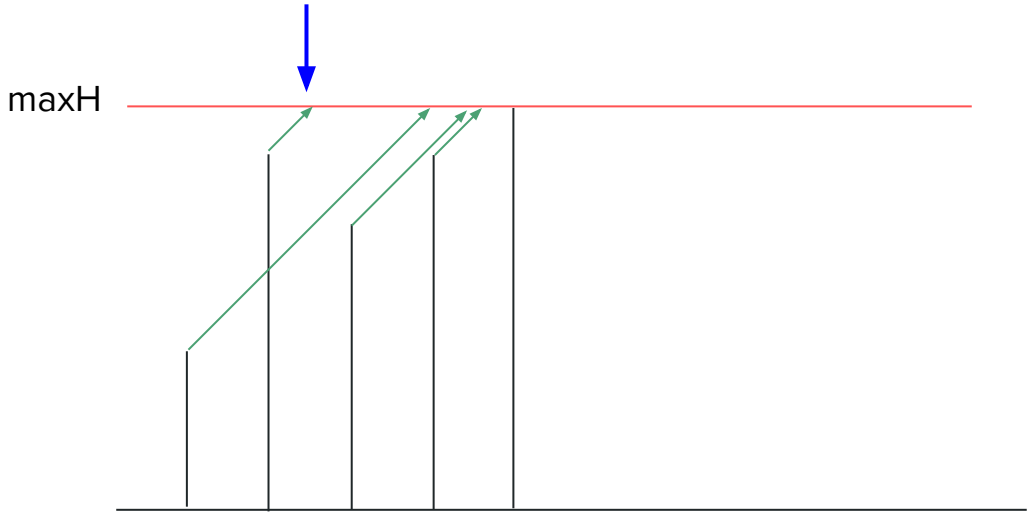
Bentuk path



- Cari kapan ia bergerak lurus
- Banyaknya gerakan naik pasti max(H[S..T]) - H[S]
- Banyaknya gerakan turun pasti max(H[S..T]) - H[T]

Cari titik terkiri

Cari titik terkiri mulai bergerak lurus



$\min(i + (maxH - H_i))$

Cari titik terkanan

- Dengan cara yang sama:
  - max(i - (maxH - H[i]))

Subtask 5

- $N, Q \leq 1000$
- Untuk tiap pertanyaan (S, T):
  - Hitung maxH := max(H[S..T])
  - Hitung L := min(i + (maxH - H[i]))
  - Hitung R := max(i - (maxH - H[i]))
  - Jawaban: 4(maxH - H[S]) + 2(R - L) + (maxH - H[T])

Solusi penuh

- Susun ulang
  - min(i + (maxH - H[i])) = min(i - H[i]) + maxH
  - max(i - (maxH - H[i])) = max(i + H[i]) - maxH
- Untuk mencari nilai max(H[i]), min(i - H[i]), dan max(i + H[i]) bisa menggunakan Segment Tree
- Kompleksitas waktu: O(Q log N)